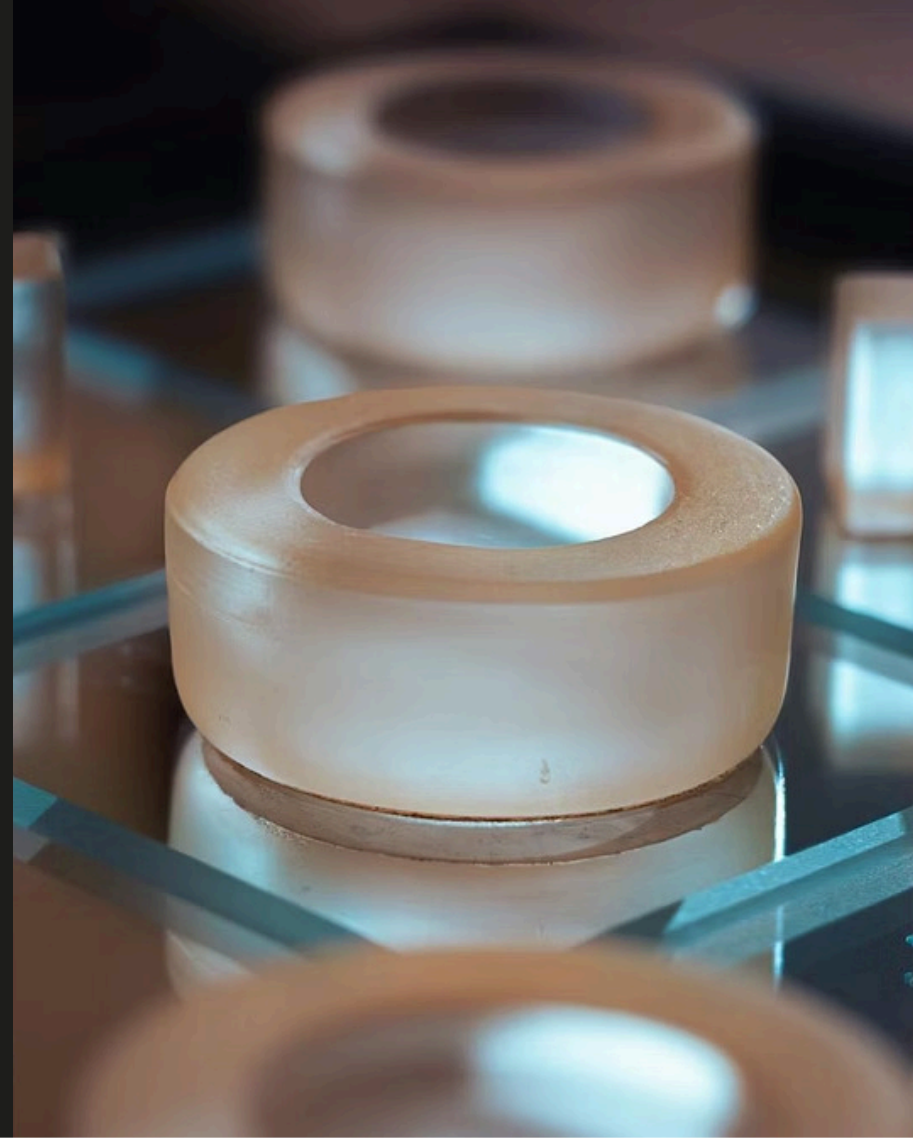
Tic-Tac-Toe Project



Team Members

- M.AKHIRA NANDAN
- S.BOSS
- A.SRIJA
- MD.SAMEEMA JAHA

Project Overview

This command-line application implements a fully functional Tic-Tac-Toe game where two players take turns marking spaces on a 3×3 grid. The system manages game state, validates moves, and detects wins or draws automatically.

Perfect for learning fundamental programming concepts including array manipulation, user input handling, and game logic implementation.

Core Game Architecture

Board Representation

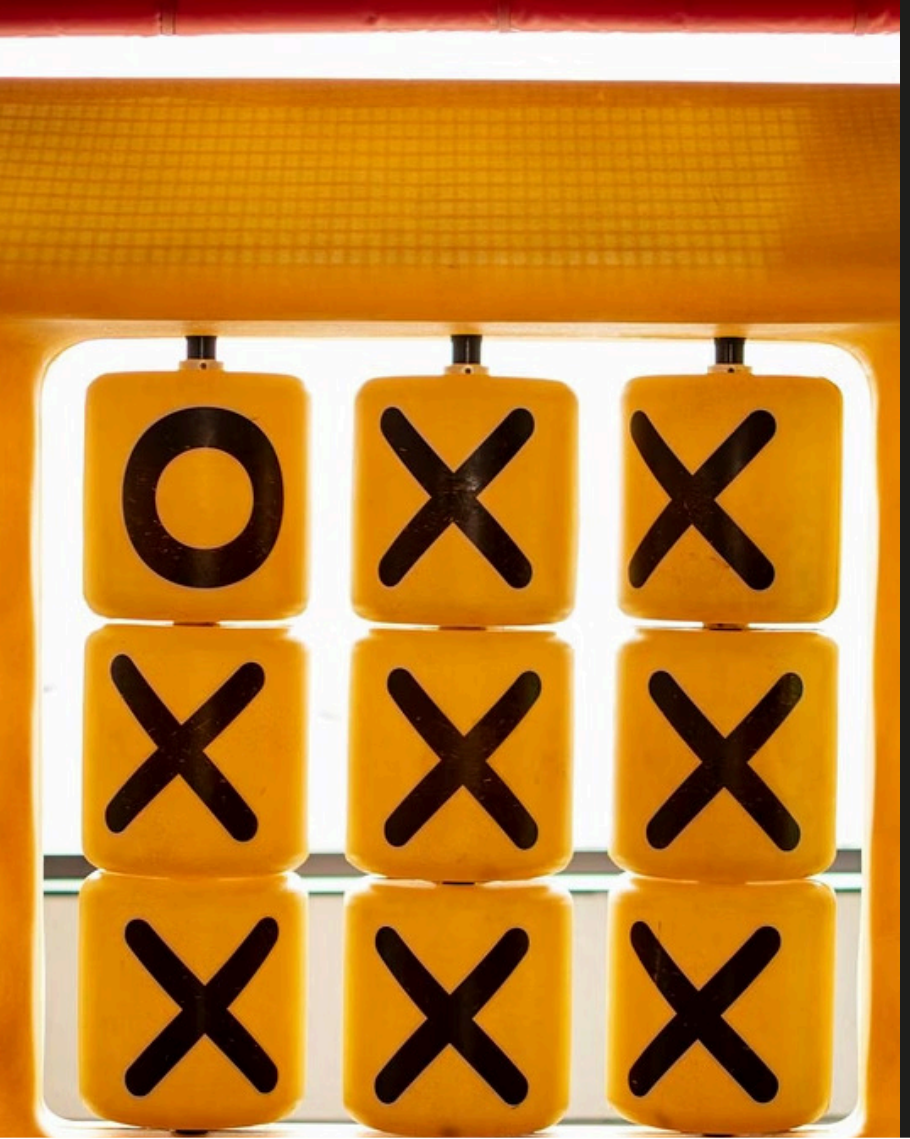
2D character array (3×3) storing game state with 'X', 'O', or empty spaces

Turn Management

Cycling logic alternates between players, ensuring fair gameplay

Win Detection

Comprehensive checks for horizontal, vertical, and diagonal victory conditions



Board Data Structure

01

Initialize 3×3 Array

Create character array with empty spaces

02

Coordinate System

Map user input (1-3) to array indices (0-2)

03

Update Board State

Place 'X' or 'O' at specified position

User Input Handling

Input Capture

Players enter row and column numbers (1-3) separated by space or comma. System parses and validates the format.

Move Execution

Convert coordinates to array indices and place the current player's mark on the board.

Input Validation Logic

Bounds Checking

Ensures row and column values fall within 1-3 range, preventing array index out of bounds errors

Occupancy Check

Verifies the selected cell is empty before allowing a move, preventing overwrite of existing marks

Format Validation

Handles invalid input formats gracefully with clear error messages and retry prompts

Win Detection Algorithm

1

Check Rows

Verify any row has three identical marks

2

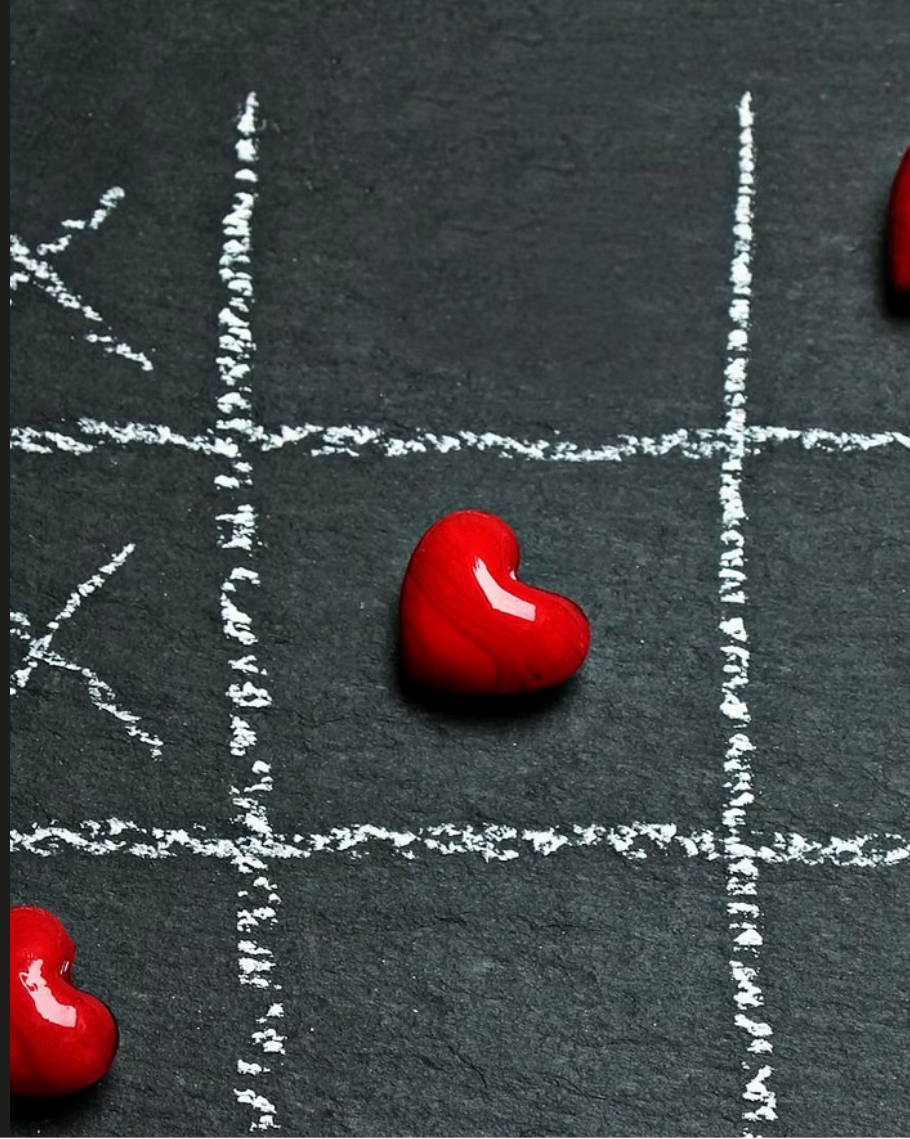
Check Columns

Verify any column has three identical marks

3

Check Diagonals

Verify either diagonal has three identical marks



Draw Condition: Cat's Game

Full Board Detection

When all nine cells are occupied and no player has achieved three in a row, the game ends in a draw.

The system counts remaining empty spaces after each move. When the count reaches zero without a winner, it declares a "Cat's Game."

- Board full check: No empty cells remain
- No winner detected: Win condition not met
- Game ends: Players must restart

Game State Flow

1

Initialize Board

Set up empty 3×3 grid and starting player

2

Player Move

Capture and validate input coordinates

3

Update Display

Refresh board with new mark shown

4

Check Win/Draw

Test for victory or full board

5

Continue or End

Repeat or declare result and exit

Key Learning Outcomes

Array Manipulation

Master 2D array indexing and state management for game boards

Input Validation

Build robust error handling and user input sanitization skills

Game Logic

Implement win detection algorithms and game state transitions

CLI Development

Create interactive command-line interfaces with clear user feedback